The Inbound Pump, the Isolate Manager, and the Option-B Decision

Feature 025 — Multi-Protocol Peer-to-Peer Link Layer · design reference

glpnet — feature 025

2026-06-07

Purpose of this document

This is a design-reference note written during Phase-3 implementation of the multi-protocol link layer (feature 025). It records, for later review:

1. **What the "pump" is** — the inbound bridge that lets an asynchronous transport feed the single-threaded GLP runtime — with a step-by-step worked example.
2. **What `isolate_manager` is** — its core purpose, its public API, every call site, and the ratified Option-C concurrency model it embodies.
3. **The Option-B decision** — why we add a generic inbound pump to the engine rather than reuse the isolate event loop, and how the two relate.

It is subordinate to the binding contracts under `specs/025-multi-protocol-link-layer/contracts/`; where this note and a contract disagree, the contract wins.

Part 1 — The single-threaded runtime and why ingress needs a pump

1.1 The loop machinery that already exists

The GLP runtime runs goals through three nested levels. None of them changes for the link layer:

- `runner.RunWithStatus(cx)` — one goal's bytecode loop: `while pc < ops.Count { dispatch op; advance/jump/stop }` (`out/csharp/lib/bytecode/runner.cs`). There is **no** message-pump step inside it; you do not check the network between bytecode instructions.
- `scheduler.DrainWithStatus(...)` — drains the goal queue to quiescence: `while Gq not empty { dequeue goal; RunWithStatus(it) }` (`out/csharp/lib/runtime/scheduler.cs:418`). A suspended goal parks in the heap suspension list (it leaves `Gq`); a reactivated goal is re-enqueued into `Gq`.
- **The driver above the scheduler** — for the interactive REPL this is the read-eval-print loop; for in-process multi-agent it is the isolate event loop (Part 2).

The key invariant: **the heap (`HeapFCP`) is single-threaded and lock-free.** Every mutation happens on one owner thread. This is the ratified *Option-C* model (Part 2.4).

1.2 What the REPL's loop actually watches

The REPL is a loop, but it watches **stdin only** (`out/csharp/bin/glp_repl.cs:117`):

```
while (true) {
    Console.Write("GLP> ");
    var input = Console.ReadLine();   // blocks here, on stdin
    ...                               // load a file, or run a goal to quiescence, print
}
```

When you type a goal it runs a **synchronous drain to quiescence** (`scheduler.DrainWithStatus`), prints `succeeds` / `suspended` / `fails`, and returns to `ReadLine()`. Its only event source is the human typing. There is **no provision to service a second, asynchronous input source (a socket) concurrently.**

1.3 The "pump" defined

A **pump** is a single step that moves items across the thread boundary at a safe point:

- the **background network thread** decodes an arriving frame and does a thread-safe **enqueue** into an `inbox` (it never touches the heap);
- the **pump**, running **on the runner thread**, **dequeues** one item and applies it via `MadContext.HandleMadAssignment(` which binds heap cells and pushes reactivated goals into `Gq`; the scheduler then runs them.

  From an `inbox` (a thread-safe queue filled by `RecvBytesAsync` -> reassemble -> order) **to** the runtime (`HandleMadAssignment` -> heap bind -> `Gq`), executed **on the runner thread**.

The `inbox` is the *only* structure shared across threads — exactly the same discipline Option-C already uses (its per-agent `Channel`).

1.4 The asymmetry: egress rides the drain, ingress needs the pump

- **Egress (sending) needs no pump.** The GLP program writing `Out` happens *on the runner thread*, so a registered `Heap.OnBind(OutWriter, …)` callback fires synchronously inside the ordinary drain. The callback extracts the value, frames it, and hands it to `SendBytesAsync` — only the socket write itself is async.
- **Ingress (receiving) is the sole reason the pump exists.** Frames arrive on a *background* thread, must be applied *on the runner thread*, and must **re-enter a scheduler that has already gone quiescent.** Nothing in today's REPL re-enters the scheduler except a new stdin line.

1.5 Why a partial goal stalls without the pump

When one program is split across two instances, each instance holds **half the goals**. Each half drains to *suspension* on the shared link variable and then has nothing to run — `Gq` is empty, so the scheduler returns. The link frames are the missing fuel. **The pump re-injects arrived link data into `Gq`, so the otherwise quiescent scheduler has work again** — "making the loop executable and moving."

1.6 Worked example — `producer` (A) -> `consumer` (B) over one link

Setup: A is the connector with `Link = ch(In?, Out)`, producing ground values onto `Out`; B is the listener with `Link = ch(In?, Out)`, consuming from `In` via `link_recv`. A's `Out` feeds B's `In`.

Instance A — egress, entirely within the existing drain

```
A1. Goal produce(Link) enqueued -> Gq = [produce]
A2. DrainWithStatus runs produce, which binds Out = [v1 | Out']   (head writer-construction)
A3.   binding Out fires Heap.OnBind(OutWriter) ON THE RUNNER THREAD:
        - extract v1 from the cons cell
        - ground-relay gate ok -> FrameCodec.Encode(v1, seq=0)
        - SendWindow.Acquire (credit) -> endpoint.SendBytesAsync(frame)   <- only THIS is async
        - re-arm OnBind on Out''s writer for the next value
A4. produce recurses -> Out' = [v2 | Out''] -> OnBind fires again -> ships v2 ...
```

```
A5. produce ends (or suspends on backpressure) -> Gq empty -> drain returns
```

A never needed a pump: every send was triggered by the program writing, on the runner thread, during the ordinary drain.

Instance B — ingress, the part that needs the pump

```
B1. Goal consume(Link) enqueued -> Gq = [consume]
B2. DrainWithStatus runs consume:
        link_recv(Msg?, ch([Msg|In], Out?), _) needs In = [Msg | In']
        In is UNBOUND (no frame yet) -> goal SUSPENDS on the In-stream head reader
        -> parked in the heap suspension list, NOT in Gq
B3. Gq is now EMPTY -> DrainWithStatus returns.

  *** FAILURE POINT WITHOUT A PUMP ***
  the REPL prints "suspended" and returns to ReadLine().
  The frame for v1 arrives on B's socket... and nothing is watching. Dead.

  WITH the pump instead:
B4. Driver loop: Gq empty + link still open -> inbox.WaitOrPoll(timeout) blocks.
B5. Background recv thread: endpoint.RecvBytesAsync -> ParseFrame -> Reassembler
        -> InboundOrdering(seq) -> decoded assignment -> inbox.Enqueue(v1-assignment)
B6. inbox.WaitOrPoll returns v1 -> PUMP applies it ON THE RUNNER THREAD:
        HandleMadAssignment-equiv: extend the In stream
        - allocate fresh pair (Wn, Rn)
        - build cons [v1 | VarRef(Rn)]
        - activations = Heap.BindVariable(InWriter, cons)   <- wakes the suspended consumer
        - foreach act: EnqueueReactivatedGoal(act) -> Gq = [consume]   <- LOOP HAS WORK AGAIN
        - advance the In cursor to Wn for the next frame
B7. Driver loops -> DrainWithStatus: consume runs:
        head unifies In = [v1 | In'],  Msg = v1 delivered to the caller
        consume recurses -> link_recv on In' -> In' unbound -> SUSPENDS again
B8. Gq empty -> drain returns -> driver waits on inbox for v2 -> (back to B5)
...
Bn. A closes Out (binds []) -> a "closed" frame arrives -> pump applies it ->
        consume's [] clause reduces (eos) -> no re-suspend ->
        link closed, Gq empty, inbox empty, no open links -> termination -> driver exits
```

The only genuinely new piece is **B4**–**B6** (block-on-inbox + apply-on-runner-thread).

1.7 Impacts, effects, side-effects

1. **Lifecycle change only for linked programs.** With no pump set, or no open link, the loop exits immediately after the first drain -> **identical behavior to today, zero regression** for ordinary programs.
2. **Determinism preserved (a good side-effect).** Inbound is applied only between drains, never mid-goal; each goal runs atomically to suspension/completion, then one inbound frame binds, then woken goals run. Single-owner heap invariant intact; a frame arriving mid-drain simply waits in `inbox` (no preemption).
3. **Termination becomes explicit logic.** End-of-run = all links closed/`permFail` **and** `Gq` empty **and** `inbox` empty. Risk: a wrong condition hangs on `inbox`. Mitigation: idle timeout + the existing `:limit` reduction budget.
4. **It also unblocks backpressure.** `SendWindow` credits are released by acks that arrive *inbound* -> applied at the pump, resuming a producer suspended on a full window.

5. **`FlushMessages` timing.** Outbound accumulates in `mp` during a drain, flushed each iteration — matches the isolate model exactly.
6. **Core-edit / parity cost.** The driver loop lives in the engine/REPL run code (mirrored from Dart); the `inbox` is the one new cross-thread structure and must be behaviour-mirrored Dart<->C#.

Part 2 — The Isolate Manager

2.1 Core purpose

`isolate_manager` (`out/csharp/lib/multiagent/isolate_manager.cs`, `glp_runtime/lib/multiagent/isolate_manager` is the **in-process multi-agent coordinator** for madGLP. It runs **N GLP agents**, each in its own `Task` (the C# analogue of a Dart isolate) with its **own `GlpRuntimeEngine`, heap, and `MadContext`**, and **routes cross-agent messages** between them over per-agent channels. It is the in-process *transport substrate* — and it is exactly what feature 025's `LinkTransport` seam generalises: "a `LinkTransport` leaf replaces `IsolateManager`'s `SendPort` routing with real bytes."

2.2 Message types (`IsolateMessage`)

Type	Fields	Meaning
`Ready`	`AgentId`, `SendPort` (this agent's channel writer)	agent booted; hands its inbox writer back to the manager
`Start`	—	begin executing (initial drain+flush)
`NetworkMsg`	`From`, `To`, `Payload` (bytes), `Type` (Assignment / AgentMessage)	one cross-agent message
`UIEvent`	`AgentId`, `Payload`	external (Flutter) UI input — currently unused (actors are internal)

2.3 Public API (the manager)

Member	What it does
`Boot(config, traceConfig)`	Starts the single main-port consumer **first** (so no `Ready` is lost), then spawns one `Task` per agent directive; waits until all agents report `Ready`.
`Start()`	Sends a fresh `Start` to every agent -> each does its initial `DrainWithStatus` + `FlushMessages`.
`InjectUIEvent(agentId, term)`	Test/Flutter helper: serialises a term and writes a `UIEvent` to that agent's channel. Unknown agent -> warn + return (no throw).
`Shutdown()`	Completes the main port (the consumer drains and exits) and clears agent ports. **Does NOT kill agent tasks** — "termination is external."
`OnUIOutput`	Optional callback for UI output from agents (Flutter integration).

Internal routing:

- `_HandleMessage(msg)` — `Ready` -> record the agent's channel writer in `_agentPorts`; `NetworkMsg` -> `_RouteNetworkMessage`.

- `_RouteNetworkMessage(msg)` — look up `_agentPorts[msg.To]` and `targetWriter.TryWrite(msg)` (thread-safe hand-off to the destination agent).

2.4 The ratified Option-C concurrency model

From `isolate_manager.cs` (Option C, ratified by Gabi 2026-05-21, commit `12a468f5`):

- Each agent is a **single `Task.Run`** consuming a **per-agent `Channel.CreateUnbounded<IsolateMessage>`** via `await foreach (var msg in reader.ReadAllAsync())`.
- The main-side consumer is **also a single `Task.Run`** with `await foreach` on `_mainPort.Reader`.
- **No locks, no `ConcurrentDictionary`** — the single-owning-context invariant. The unbounded `Channel` is the one thread-safe hand-off point; everything else (heap, `Gq`, `MadContext`) is single-threaded per agent.

2.5 The agent task entry point (`_AgentIsolateEntry`)

Each agent task does, in order:

1. Create a `GlpEngine` (the one way to run GLP); `EnableMadGlp(agentId)` (loads madPredicates, creates `MadContext`).
2. Load program source (project-linking or sequential).
3. Initialise the **permanent index-0 serializer entry** — the agent's network input stream `N_p` (`Wp.InitializeSerializerEntry(netInWriter)`), per madGLP spec §4.1.
4. Wire **outbound**: `ctx.OnMessageReady = (dest, msg) => config.MainPort.TryWrite(new NetworkMsg(agentId, dest, msg.Payload, msg.Type))`.
5. Find the goal label (FATAL-hang if missing: returns **without** `Ready`, deliberately hanging boot — the documented fatal-boot signal).
6. Build the goal's argument map (arg 0 = agent id; middle args = boot constants; last arg = the network-input reader) and enqueue the goal.
7. Signal `Ready` (handing back its own channel writer).
8. Run the **event loop** (this is the isolate-path analogue of the pump):

```
await foreach (var msg in selfReader.ReadAllAsync())
{
    if (msg is Start) {
        scheduler.DrainWithStatus(...); ctx.FlushMessages();
    }
    else if (msg is NetworkMsg netMsg) {
        if (netMsg.Type == Assignment) {
            var (globalName, value) = serializer.DeserializeGlobalSendPayload(...);
            ctx.HandleMadAssignment(globalName, value, netMsg.From);   // ON the agent task
        }
        scheduler.DrainWithStatus(...); ctx.FlushMessages();
    }
    else if (msg is UIEvent) { /* not processed */ }
}
```

Note **`HandleMadAssignment` is called on the agent task**, synchronously in the loop — never from a background thread. This is precisely the discipline the pump reproduces for the REPL/socket case.

2.6 Every call site (uses)

Caller	Use
`out/csharp/bin/glp_repl.cs`	the `:boot <bootfile.glp> [timeoutSec]` REPL command — runs a multi-isolate play

Caller	Use
`out/csharp/lib/engine/glp_engine.cs`	high-level engine wrapper that boots/drives agents
`out/csharp/lib/bytecode/runner.cs`	references the message types (NetworkMsg / MessageType)
`out/csharp/lib/multiagent/global_writers_table.cs`	references shared multiagent types
`out/csharp/test/multiagent/isolate_manager_test.cs`	direct manager unit tests
`out/csharp/test/multiagent/{bonds_v2,cssn_v2,multiagent_glp,multiagent_modules}_*_test.cs`	multiagent / v2 integration tests

(The Dart side mirrors these: `bin/glp_repl.dart`, `lib/engine/glp_engine.dart`, and the parallel `test/multiagent/*` suites.)

Part 3 — The Option-B decision and how it relates to the isolate manager

3.1 The fork

The link layer's background `RecvBytesAsync` must reach `HandleMadAssignment` on the single-threaded runner. Three ways:

Option	What	Consequence
A. Reuse the isolate Channel-inject	recv loop `TryWrite`s a `NetworkMsg` into the agent's inbound `Channel`; the agent event loop applies it	reuses the ratified model **as-is**, but **only the isolate path has that channel** — the Phase-4 two-REPL split has no event loop, and the channel is SendPort/in-process, not a cross-process socket
B. Add a runner-drained inbound pump to the engine *(CHOSEN)*	a thread-safe `inbox` + an `IInboundPump` seam the engine driver drains each cycle: `drain -> flush -> while pump.HasPendingOrLive: TryApplyNext -> drain`	works for **both** REPL and isolate paths uniformly; a small, careful core edit to `out/csharp` (mirrored to Dart) at the run-loop boundary
C. Cooperative pump from the REPL loop	the kernel installs a pump the existing REPL loop calls	least new machinery *if* the REPL loop has an extension point; still edits `glp_repl.cs`

Decision: Option B (Gabi, 2026-06-07).

3.2 Why B, given the isolate manager already has a loop

The isolate manager's event loop (Part 2.5) is the *right idea* but the *wrong reach* for feature 025:

- it routes between isolates **in one process** over Dart `SendPort`s / C# `Channel`s — not between **two processes** over a **socket**;
- the Phase-4 headline split runs **two REPL processes**, which use the synchronous `DrainWithStatus` path and **have no isolate event loop at all**.

Option B adds the inbound `inbox` + pump + driver-loop to the **engine** so the *same* mechanism serves the REPL split, the Dart<->C# parity gate, and (later) the isolate path — rather than the link layer behaving differently depending on which host drives it.

3.3 Why B is consistent with Option C (not a violation)

Option B **extends** Option C's existing principle rather than breaking it:

- Option C's safety rests on **one thread-safe hand-off structure** (the per-agent `Channel`) plus **apply-on-owner-thread** (`HandleMadAssignment` runs on the agent task).
- Option B's `inbox` is exactly that one thread-safe hand-off structure for the socket case, and the pump applies **on the runner thread**. No new locking on the heap; the single-owning-context invariant holds.

3.4 Planned shape of B (for review)

- **Core seam (`out/csharp`, C#-first):** an `IInboundPump` interface in the runtime (`HasPendingOrLive`, `TryApplyNext(timeout)`); an optional `engine.InboundPump` field; the goal run-to-quiescence becomes the driver loop above. With no pump set the loop collapses to today's single drain — zero change for non-link programs.
- **Link side (`csharp/glp_link`):** implements `IInboundPump` — owns the `inbox` and the background recv loops, knows the open-link count, and in `TryApplyNext` dequeues one decoded assignment and calls `MadContext.HandleMadAssignment` on the runner thread.
- **Dependency direction:** `glp_link -> out/csharp` only (the interface lives in the engine; the link layer implements it) — no circular reference, no clobber of hand-authored link code.
- **Parity:** mirror the engine pump to Dart in Phase 8.

Appendix — key file:line anchors

- REPL stdin loop: `out/csharp/bin/glp_repl.cs:117-120`.
- Scheduler drain: `out/csharp/lib/runtime/scheduler.cs:418` (`DrainWithStatus`).
- Runner bytecode loop: `out/csharp/lib/bytecode/runner.cs` (`RunWithStatus`).
- Inbound apply: `out/csharp/lib/multiagent/mad_context.cs` (`HandleMadAssignment`), the serializer stream-extend idiom (cold-call assignment handler).
- Outbound seam: `MadContext.OnMessageReady`, `FlushMessages`.
- Egress observe: `HeapFCP.OnBind(writerAddr, callback)`.
- Isolate manager: `out/csharp/lib/multiagent/isolate_manager.cs` (class `:234`, `Boot :273`, `Start :334`, `Shutdown :372`, `_RouteNetworkMessage :401`, `_AgentIsolateEntry :445`, agent event loop `:564-620`).